

EEE3096S 2025 Practical 2 Code

Phila Haanyama (HNYPHI001) & Lulama Lingela (LNGLUL002)

Github Repo Link: <https://github.com/LlamaLuLu/EEE3096S-Practicals-Group-34.git>

```
/*
 * assembly.s
 *
 */

@ DO NOT EDIT
.syntax unified
.text
.global ASM_Main
.thumb_func

@ DO NOT EDIT
vectors:
    .word 0x20002000
    .word ASM_Main + 1

@ DO NOT EDIT label ASM_Main
ASM_Main:

    @ Some code is given below for you to start with
    LDR R0, RCC_BASE          @ Enable clock for GPIOA and B by setting bit 17 and 18 in
RCC_AHBENR
    LDR R1, [R0, #0x14]
    LDR R2, AHBENR_GPIOAB    @ AHBENR_GPIOAB is defined under LITERALS at the end of the
code
    ORRS R1, R1, R2
    STR R1, [R0, #0x14]

    LDR R0, GPIOA_BASE        @ Enable pull-up resistors for pushbuttons
    MOVS R1, #0b01010101
    STR R1, [R0, #0x0C]
    LDR R1, GPIOB_BASE @ Set pins connected to LEDs to outputs
    LDR R2, MODER_OUTPUT
    STR R2, [R1, #0]
    MOVS R2, #0               @ NOTE: R2 will be dedicated to holding the value on the LEDs

@ TODO: Add code, labels and logic for button checks and LED patterns
MOVS R5, #0                  @ initialize saved pattern
MOVS R6, #0                  @ initialize SW2 previous state (0 = not pressed)

main_loop:
    @ ----- check inputs -----
    LDR R3, [R0, #0x10]          @ load IDR into R3

    @ ----- freeze control (SW3) -----
check_SW3:
    MOVS R4, #8                  @ load bit 3 into R4
    TST R3, R4                  @ bitwise AND
    BEQ SW3_pressed
    @ SW3 not pressed:
    B check_SW2                 @ continue

SW3_pressed:
    @ just freeze (don't change R2)
```

B write_LEDs @ skip all logic

@ ----- preset LED pattern control (SW2) -----

check_SW2:

MOVS R4, #4 @ load bit 2 into R4
TST R3, R4 @ bitwise AND
BEQ SW2_currently_pressed

@ SW2 not currently pressed (check if just released)

CMP R6, #1 @ SW2 previously pressed?

BNE skip_restore @ if already not pressed, don't restore

@ SW2 just released (restore saved value)

MOVS R2, R5 @ restore saved pattern

MOVS R6, #0 @ set to "not pressed"

B write_LEDs @ skip increment

skip_restore:

@ SW2 now not pressed (normal increment)

MOVS R6, #0 @ set to "not pressed"

B check_SW0 @ increment logic

SW2_currently_pressed:

@ check for new press (edge detection)

CMP R6, #0 @ SW2 previously not pressed?

BNE skip_save @ if already pressed, don't save again

@ 1st press (save current incrementing pattern)

MOVS R5, R2 @ save current pattern

MOVS R6, #1 @ set to "pressed"

skip_save:

@ while SW2 pressed (load preset pattern)

MOVS R2, #0xAA @ load preset LED pattern

B write_LEDs @ skip increments

@ ----- LED increment logic (SW0: while SW2 not pressed) -----

check_SW0:

MOVS R4, #1 @ load bit 0 into R4

TST R3, R4 @ bitwise AND

BEQ SW0_pressed

@ if SW0 not pressed:

ADDS R2, R2, #1 @ increment LEDs by 1

B write_LEDs

@ if SW0 pressed:

SW0_pressed:

ADDS R2, R2, #2 @ increment LEDs by 2

@ ----- output LEDs -----

write_LEDs:

STR R2, [R1, #0x14] @ write to LEDs

@ ----- delay logic (SW1) -----

check_SW1:

MOVS R4, #2 @ load bit 1 into R4

TST R3, R4 @ bitwise AND

BEQ SW1_pressed

@ if SW1 not pressed:

LDR R4, LONG_DELAY_CNT @ load long delay

B delay_loop

@ if SW1 pressed:

SW1_pressed:

LDR R4, SHORT_DELAY_CNT

@ load short delay

delay_loop:

SUBS R4, R4, #1

@ subtract 1 from R4, update flags

BNE delay_loop

@ loops until count = 0

B main_loop

@ LITERALS; DO NOT EDIT

.align

RCC_BASE: .word 0x40021000

AHBENR_GPIOAB: .word 0b110000000000000000

GPIOA_BASE: .word 0x48000000

GPIOB_BASE: .word 0x48000400

MODER_OUTPUT: .word 0x5555

@ TODO: Add your own values for these delays

LONG_DELAY_CNT: .word 1400000 @ loops: $0.7 \text{ s} * 8 \text{ MHz} / 4 \text{ cycles}$

SHORT_DELAY_CNT: .word 600000 @ loops: $0.3 \text{ s} * 8 \text{ MHz} / 4 \text{ cycles}$